

Automação e otimização de Sistemas Android Embarcados

Objetivo

- Utilizar ferramentas de automação para implantar aplicativos Android em emuladores ou dispositivos;
- Explicar a configuração do kernel Linux para habilitar o suporte à rede CAN;
- Explicar as principais estratégias de otimização de código em desenvolvimento de software;
- Justificar o uso de estratégias de otimização de código em sistemas embarcados Android automotivos.

Repositório

Link do Repositório: [gitHub](#)

Link da Aplicação: [VehicleEqualizerFull](#)

Link do Vídeo: [Vídeo](#)

Sumário

1. [Ambiente de desenvolvimento](#)
 - 1.1. [Sistema Operacional](#)
 - 1.2. [Java\(JDK\)](#)
 - 1.3. [Android Studio](#)
 - 1.4. [Git](#)
 - 1.5. [Emulador Android](#)
2. [Aplicativos Desenvolvidos Durante o Curso](#)
3. [Execução de scripts de automação e configuração do ambiente Android](#)
4. [Explicação e configuração do kernel Linux para habilitar o suporte à rede CAN](#)

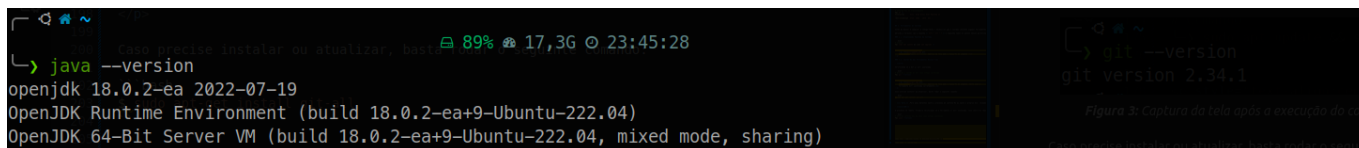
1. Ambiente de Desenvolvimento

1.1. Sistema Operacional

```
# O comando lsb_release imprime certas informações de LSB (Linux Standard Base) e distribuição.  
$ lsb_release -a  
No LSB modules are available.  
Distributor ID: Ubuntu  
Description: Ubuntu 22.04.5 LTS  
Release: 22.04  
Codename: jammy
```

1.2. Java(JDK)

```
# Retorna informações do Java caso esteja instalado
$ java --version
```

A terminal window with a dark background. The prompt is a green arrow. The command 'java --version' has been entered. The output shows 'openjdk 18.0.2-ea 2022-07-19', 'OpenJDK Runtime Environment (build 18.0.2-ea+9-Ubuntu-22.04)', and 'OpenJDK 64-Bit Server VM (build 18.0.2-ea+9-Ubuntu-22.04, mixed mode, sharing)'. In the background, a window titled 'Figura 3: Captura da tela após a execução do comando' is visible, showing the output of 'git --version' as 'git version 2.34.1'.

```
openjdk 18.0.2-ea 2022-07-19
OpenJDK Runtime Environment (build 18.0.2-ea+9-Ubuntu-22.04)
OpenJDK 64-Bit Server VM (build 18.0.2-ea+9-Ubuntu-22.04, mixed mode, sharing)
```

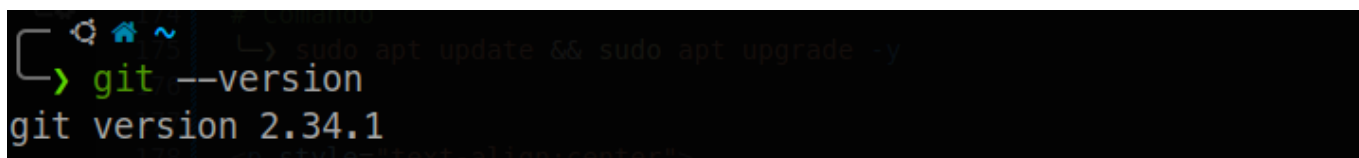
Figura 1: Captura da tela após a execução do comando, ferramenta instalada corretamente

1.3. Android Studio

```
# Informações gerais do Android Studio
Android Studio Narwhal 3 Feature Drop | 2025.1.3
Build #AI-251.26094.121.2513.14007798, built on August 28, 2025
Runtime version: 21.0.7+-13880790-b1038.58 amd64
VM: OpenJDK 64-Bit Server VM by JetBrains s.r.o.
Toolkit: sun.awt.X11.XToolkit
Linux 6.8.0-83-generic
Ubuntu 22.04.5 LTS; glibc: 2.35
Kotlin plugin: K2 mode
GC: G1 Young Generation, G1 Concurrent GC, G1 Old Generation
Memory: 2968M
Cores: 12
Registry:
  ide.experimental.ui=true
Current Desktop: ubuntu:GNOME
```

1.4. Git

```
# Retorna a versão do Git caso esteja instalado
$ git --version
```

A terminal window with a dark background. The prompt is a green arrow. The command 'git --version' has been entered. The output is 'git version 2.34.1'. In the background, a window titled 'Figura 2: Captura da tela após a execução do comando' is visible, showing the output of 'git --version' as 'git version 2.34.1'.

```
git version 2.34.1
```

Figura 2: Captura da tela após a execução do comando, neste caso ferramenta está instalada corretamente

1.5. Emulador Android

O Emulador Android utilizado é o mesmo da atividade anterior, não foi realizada que já foi entregue e descrita na sessão [1.5 Emulador Android](#) do relatório [Interface e Gerenciamento de Serviços no Android](#).

2. Aplicativos Desenvolvidos Durante o Curso

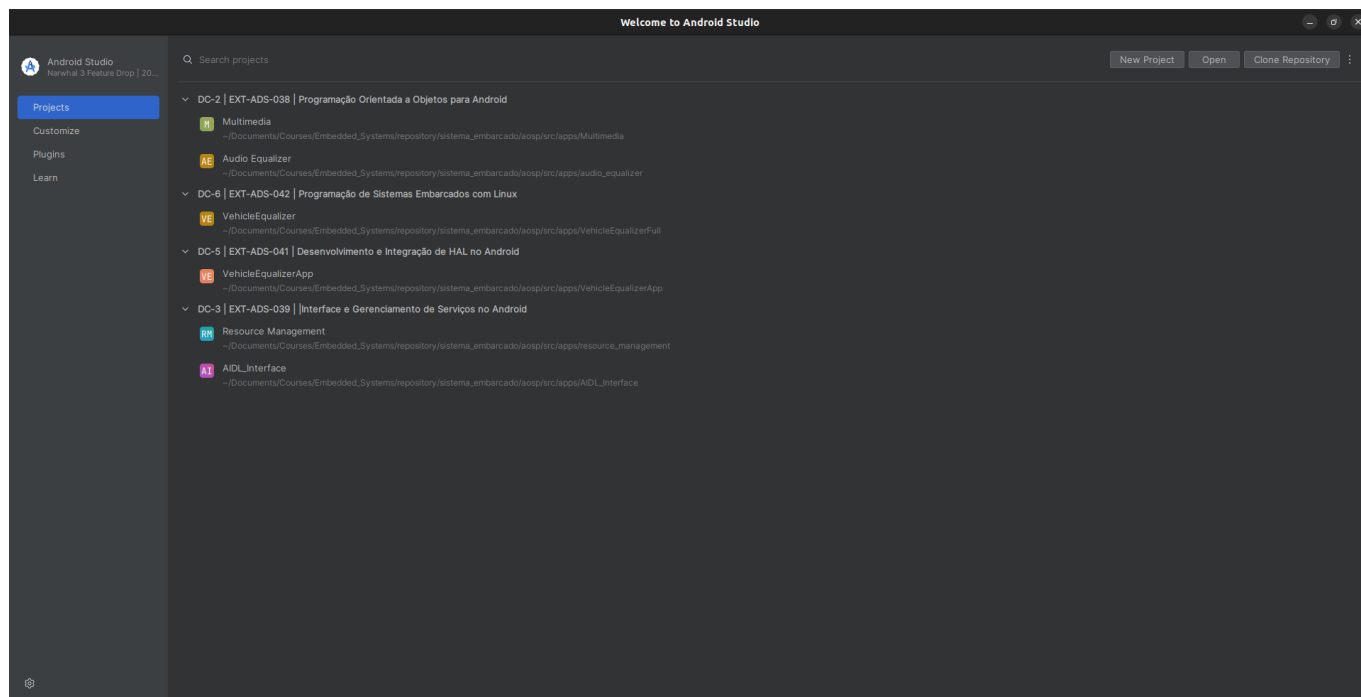


Figura 3: Aplicativos agrupados por disciplinas.

3. Execução de scripts de automação e configuração do ambiente Android

Script de implantação simples, verifica se existe algum dispositivo conectado, se sim, verifica se o aplicativo que vai ser instalado já está instalado, se sim, desinstala e reinstala a nova versão, além disso, concede permissão inicial do aplicativo para ir direto para a tela inicial, além disso, dá PLAY na música e após 5 segundos dá PAUSA. Verificar o LOG para ver a execução.

```
#!/bin/bash

# Script de implantação e teste para o aplicativo Vehicle Equalizer
# Este script automatiza a instalação, execução e coleta de logs do
aplicativo em um dispositivo/emulador Android

# Caminho para o APK do seu aplicativo

# find . -name "*.apk"
APK_PATH="app/build/intermediates/apk/debug/app-debug.apk"

# TAG para filtrar logs no Logcat (usada no seu código Kotlin/Java)
LOGCAT_TAG="VehicleEqualizerApp|AudioService|PlaybackModule"

# Nome do pacote do seu aplicativo (definido no AndroidManifest.xml)
# Retorna o caminho do pacote, caso esteja instalado.
PACKAGE_NAME=$(adb shell pm list packages | grep vehicleequalizer | sed
's/package:/' )

echo "=====
```

```

echo "Iniciando script de implantação e teste do Equalizador..."
echo "=====

# 1. Verificar se um dispositivo/emulador está conectado e online
echo "Verificando dispositivos Android conectados..."
if ! adb devices | grep -q "device";
then
    echo "ERRO: Nenhum dispositivo ou emulador Android encontrado. "
    echo "Certifique-se de que um emulador esteja rodando ou um dispositivo
esteja conectado e online (modo depuração USB ativado)."
exit 1
fi
echo "Dispositivo/emulador encontrado e online."

# 2. Desinstalar a versão anterior do aplicativo (se existir) para garantir
uma instalação limpa\
# Só tenta desinstalar se o pacote for encontrado
if [ -n "$PACKAGE_NAME" ]; then
    echo "Desinstalando versão anterior do aplicativo ($PACKAGE_NAME)..."
    adb uninstall $PACKAGE_NAME
else
    echo "Nenhum pacote encontrado com 'vehicleequalizer'."
fi

# 3. Instalar o novo APK
echo "Instalando o APK: $APK_PATH..."
if [ ! -f "$APK_PATH" ]; then
    echo "ERRO: APK não encontrado em $APK_PATH. Por favor, construa o
projeto primeiro (ex: ./gradlew assembleDebug)."
exit 1
fi
# Adição de -t para permitir a instalação de APKs de teste (debug)
# Isso evita o erro: "Failure [INSTALL_FAILED_TEST_ONLY: install
adb install -t "$APK_PATH"

# Verifica o código de saída do comando 'adb install'
if [ $? -ne 0 ]; then
    echo "ERRO: Falha na instalação do APK. Verifique o caminho do APK e
as permissões."
exit 1
fi
echo "Aplicativo instalado com sucesso."

# 4. Iniciar o aplicativo (Activity principal)
echo "Iniciando o aplicativo $PACKAGE_NAME..."
adb shell am start -n "$PACKAGE_NAME/.ui.MainActivity"

# # 5. Esperar um pouco para o aplicativo iniciar completamente e a UI
carregar
echo "Aguardando 5 segundos para o aplicativo iniciar..."
sleep 5

# Verifica se está na tela de permissão e concede a permissão de
notificações se necessário

```

```

echo "Verificando se a permissão de notificações precisa ser concedida..."
CURRENT_APP=$(adb shell dumpsys window | grep -E 'mCurrentFocus' | sed
's/.* //g')
if [[ "$CURRENT_APP" == *"com.google.android.permissioncontroller"* ]];
then
    adb shell pm grant "$PACKAGE_NAME"
android.permission.POST_NOTIFICATIONS
else
    echo "A permissão de notificações já foi concedida"
fi

# # 6. Limpar o Logcat para capturar apenas logs relevantes do teste atual
# echo "Limpando Logcat..."
# adb logcat -c

# # 7. Simular interações básicas e capturar logs
# echo "Simulando interação: Ativando e desativando o equalizador..."

# 7. Envia comando via adb para da play na musica via AudioService
echo "Iniciando o serviço de áudio para simular reprodução..."
adb shell am start-foreground-service -n
$PACKAGE_NAME/.service.AudioService -a $PACKAGE_NAME.ACTION_PLAY
sleep 5 # Espera 5 segundos com a música tocando
echo "Parando a reprodução de áudio..."
adb shell am start-foreground-service -n
$PACKAGE_NAME/.service.AudioService -a $PACKAGE_NAME.ACTION_STOP
sleep 2 # Espera 2 segundos para garantir que o serviço pare

echo "Coletando logs relevantes do Logcat (TAG: $LOGCAT_TAG)..."
# Coleta logs da MainActivity e AudioService
adb logcat -v time -d | grep -E $LOGCAT_TAG > app_logs.txt
echo "Logs salvos em app_logs.txt no diretório do projeto."

# 8. Verificar se o aplicativo está rodando (opcional)
echo "Verificando se o processo do aplicativo está ativo..."
if adb shell ps | grep -q $PACKAGE_NAME; then
    echo "Processo do aplicativo $PACKAGE_NAME está ativo."
else
    echo "Processo do aplicativo $PACKAGE_NAME NÃO está ativo."
fi

echo "=====
echo "Script de implantação e teste concluído."
echo "Verifique 'app_logs.txt' para os logs coletados."
echo "=====

```

Arquivo de saída resultando: `app_logs.txt`

```

D/PlaybackModule(19938): MediaPlayer released
D/PlaybackModule(19938): Playback started.
D/PlaybackModule(19938): MediaPlayer prepared
D/PlaybackModule(19938): AudioSessionId requested: 3401

```

```

D/AudioService(19938): Equalizer initialized with sessionId=3401
D/PlaybackModule(19938): Current position: 0 ms
D/PlaybackModule(19938): Current position: 40 ms
D/PlaybackModule(19938): Current position: 986 ms
D/PlaybackModule(19938): Current position: 1144 ms
D/PlaybackModule(19938): Current position: 2148 ms
D/PlaybackModule(19938): Current position: 3150 ms
D/PlaybackModule(19938): Current position: 4151 ms
D/PlaybackModule(19938): Current position: 5154 ms
D/PlaybackModule(19938): Current position: 6156 ms
D/PlaybackModule(19938): Current position: 7158 ms
D/PlaybackModule(19938): Current position: 8161 ms
D/PlaybackModule(19938): Current position: 9163 ms
D/PlaybackModule(19938): Current position: 10165 ms
D/PlaybackModule(19938): Current position: 11168 ms
D/PlaybackModule(19938): Paused playback.
D/PlaybackModule(19938): Current position: 11489 ms
~~~~~

```

4. Explicação e configuração do kernel Linux para habilitar o suporte à rede CAN

4.1. Descrição da configuração de parâmetros do kernel e dos módulos de suporte ao CAN, incluindo o processo de compilação e carregamento

- Para evitar que a compilação quebre, instalei essas dependências:

```

```bash
sudo apt update
sudo apt install git repo curl bc build-essential flex bison libssl-dev
libncurses5-dev libncursesw5-dev u-boot-tools device-tree-compiler

```

- A primeira etapa é baixar o kernel do Android. Para baixar, eu seguir os passos abaixo, estou utilizando a versão 13.

```

Criei uma pasta no meu diretório local
mkdir -p ~/android-kernel
cd ~/android-kernel

Inicializando o Repo
repo init -u https://android.googlesource.com/kernel/manifest -b android13-5.15

Sincronizando o repositório
repo sync -c -j$(nproc)

```

- A segunda etapa é habilitar os módulos de kernel CAN.

```
cd common-android13-5.15/common
```

```
Abrindo o menuconfig do kernel
```

```
make ARCH=x86_64 menuconfig
```

```
Essas são as configurações para habilitar os módulos CAN
```

```
Estou habilitando os módulos e criando os .ko não embutidos no kernel,
caso queira, basta carregar apenas os módulos no dispositivo alvo
```

```
#
```

```
Networking support --->
```

```
Networking options --->
```

```
CAN bus subsystem support --->
```

```
CAN bus subsystem
```

```
[M] CAN bus subsystem
```

```
<*> CAN RAW protocol
```

```
[M] Virtual CAN interface (vcan)
```

```
depois de habilitado, fiz o build
```

```
make ARCH=x86_64 CROSS_COMPILE= modules -j$(nproc)
```

```
E esses foram os módulos gerados
```

```
↳ find -name "*c*.ko"
```

```
./net/can/can-bcm.ko # CAN Broadcast Manager: gerencia mensagens CAN de
broadcast, filtrando e enviando mensagens de forma eficiente.
```

```
./net/can/can.ko # Módulo principal CAN core: fornece a
infraestrutura básica para suporte a CAN no kernel.
```

```
./net/can/can-gw.ko # CAN Gateway: permite rotear mensagens entre
diferentes redes CAN.
```

```
./net/can/can-raw.ko # CAN Raw Sockets: fornece interface de baixo
nível para enviar e receber frames CAN diretamente do espaço do usuário.
```

```
./drivers/net/can/dev/can-dev.ko # Driver genérico de dispositivos CAN:
abstrai hardware específico, fornecendo interface unificada para
dispositivos CAN.
```

```
./drivers/net/can/vcan.ko # Virtual CAN (vcan): simula uma rede CAN
no kernel para testes e desenvolvimento sem hardware físico.
```

- Para carregar esses módulos em um dispositivo real sem mudar o kernel completo, basta executar os comandos abaixo:

```
adb root
```

```
copia arquivos para o dispositivo
```

```
adb push commom/net/can/can-bcm.ko /data/local/tmp/
```

```
adb push commom/net/can/can.ko /data/local/tmp/
```

```
adb push commom/net/can/can-gw.ko /data/local/tmp/
```

```
adb push commom/net/can/can-raw.ko /data/local/tmp/
```

```
adb push commom/drivers/net/can/dev/can-gw.ko /data/local/tmp/
```

```
adb push commom/drivers/net/can/vcan.ko /data/local/tmp/
```

```
adb shell
```

```
su
```

```
Carrega os módulos no kernel
```

```
insmod /data/local/tmp/vcan.ko
```

```
insmod /data/local/tmp/can-raw.ko
insmod /data/local/tmp/can-bcm.ko
insmod /data/local/tmp/can.ko
insmod /data/local/tmp/can-gw.ko
insmod /data/local/tmp/can-gw.ko
```